Advanced Data Management
Quiz Refresher: Chapter 4, Answer Sheet

Answers to the 20 recall questions, the 8 multiple-choice questions and the 5 midterm-style problems

Advanced Data Management

2026-10-27

Answers

Round A

A1. An operator that must consume all of its input before emitting any output, so it materialises (and may spill to disk). Sort, hash build, hash-based aggregation (and DISTINCT) over unsorted input.

A2. GroupAggregate over sorted input: it emits each group as soon as the key changes.

A3. Zero extra I/O. They are pipelined per-tuple CPU work on tuples the child already paid to read.

A4. "...sort so they sit together, or hash so they land together." Duplicate elimination, joins (sort-merge versus hash), aggregation (GroupAggregate versus HashAggregate).

A5. Runs $= \lceil B/M \rceil$; fan-in $= M - 1$. One buffer page is reserved for the output of the merge.

A6. Passes $= 1 + \lceil \log_{M-1} \lceil B/M \rceil \rceil$. The 1 is Phase 0, run generation.

A7. About $2B\times$ passes: every pass reads and writes every page.

A8. `work_mem`. `Sort Method: quicksort Memory: ...kB` (fits) and `Sort Method: external merge Disk: ...kB` (spilled).

A9. Simple: $B_R + |R| \cdot B_S$ (inner read once per *row* of R). Block: $B_R + \lceil B_R/(M-2) \rceil \cdot B_S$ (inner read once per *chunk*). The two reserved buffers hold the current S page and the output page.

A10. $B_R + |R| \cdot (h + \text{matches})$: one index descent plus one heap read per match, per outer row.

A11. The smaller one: the outer is read once, the inner once per outer chunk, so fewer chunks means fewer inner scans.

A12. sort(R) + sort(S) + $(B_R + B_S)$. Wins when the inputs are already sorted (an index scan delivers them in order) or when sorted output is wanted anyway.

A13. Build on the smaller input; cost $B_R + B_S$, each input read exactly once,

the minimum any join can cost.

A14. When the build side does not fit in memory. Both inputs are partitioned to disk by a hash function (read and write both), then each partition pair is joined in memory: about $3(B_R + B_S)$. In the plan: `Batches` greater than 1 under `Hash`.

A15. Nested loop handles anything, since it just tests the predicate; sort-merge handles ranges. Hash cannot: hashing finds equal keys only, and $a < b$ gives no bucket to look in.

A16. a is when the first row came out, b when the last did. $a = b$ on a Hash node means it produced nothing until it was finished: the signature of a pipeline breaker.

A17. The inner side ran 20,000 times, once per outer row: index nested-loop join, one index descent per outer tuple.

A18. The number of groups, hence the size of the hash table. Few groups: HashAggregate. Groups near the row count, or input already sorted: Sort (if needed) then GroupAggregate.

A19. Forbids the hash join so the planner falls back to merge join, then (with `enable_mergejoin = off`) to nested loop. Run `RESET ALL;` afterwards or every later experiment is silently distorted.

A20. Rows come out in *probe-side file order*; the hash table has no order, and the probe simply streams the larger input as stored.

Round B

B1. **(b) 3.** Runs $\lceil 5000/51 \rceil = 99$; fan-in 50; $50 < 99 \leq 2500 = 50^2$ so 2 merge passes; $1 + 2 = 3$. (d) is the run count, not the passes.

B2. **(b) 64.** Two passes means every initial run merges at once: $\lceil 4000/M \rceil \leq M - 1$. $M = 64$: 63 runs, fan-in 63, yes. $M = 63$: 64 runs, fan-in 62, no. Rule of thumb $\sqrt{B} \approx 63.2$, then check the boundary.

B3. **(c) 80,500.** Chunks $\lceil 500/(27-2) \rceil = 20$; $500 + 20 \times 4000 = 80{,}500$. (b) uses M instead of $M - 2$ (19 chunks); (d) uses the simple nested loop with 500 *rows*; (a) forgot the rescans.

B4. **(d).** Sorted input means a group is complete the moment the key changes, so it streams. Sort, hash build and HashAggregate all need the whole input first.

B5. **(c) 62,400.** Grace: partition both (read and write, $2 \times 20{,}800$) then join each partition pair (20,800): $3 \times 20{,}800$. (a) is the in-memory cost; (d) is a simple nested loop with 800 rows.

B6. **(b).** Batches are Grace partitions. 1 means the build fit in `work_mem`; 8 means both inputs were split into 8 partitions on disk and joined pair by pair.

B7. **(c).** A hash join finds *equal* keys; an inequality has no bucket to probe. (b) is fine: hash the pair (x, z). (d) is fine: hash join needs no index.

B8. **(b).** The planner estimates one hash-table entry per group; past `work_mem` it prefers to sort and stream groups one at a time. (c) would give a HashAggregate.

Round C

Solution: C1. External sort at scale

(a) Runs = 600,000/500 = 1,200. Fan-in = 499. Merge passes = $\lceil \log_{499} 1200 \rceil = 2$ since $499 < 1200 \leq 499^2 = 249{,}001$. Passes $= 1 + 2 = 3$. I/O $\approx 2 \times 600{,}000 \times 3 = 3{,}600{,}000$ page reads and writes.
(b) Two passes means one merge pass: $\lceil 600{,}000/M \rceil \leq M - 1$. Near $\sqrt{600{,}000} \approx 774.6$. $M = 775$: $\lceil 774.2 \rceil = 775$ runs, fan-in 774, fails. $M = 776$: $\lceil 773.2 \rceil = 774$ runs, fan-in 775, works. Smallest $M = 776$. I/O drops to $2 \times 600{,}000 \times 2 = 2{,}400{,}000$.
(c) 776×8 KB = 6,208 KB ≈ 6.1 MB; set `work_mem = '8MB'` to be safe.
(d) The table fits in memory: one read, an in-memory sort, no runs written. The plan shows `Sort Method: quicksort Memory: ...kB` instead of `external merge Disk: ...kB`. About B reads (plus B writes only if the result is materialised).
(e) `work_mem` is allocated per sort or hash operation per query, not per server. Fifty concurrent queries each with two sorts at 1 GB would want 100 GB, competing with the buffer pool for the same RAM. Set it modestly and raise it per session for known heavy queries.

Solution: C2. Five ways to join

(a) $B_R + |R| \cdot B_S = 2{,}000 + 100{,}000 \times 80{,}000 \approx 8 \times 10^9$ page reads. Absurd because it rescans 80,000 pages once per *row*.
(b) Chunks of $M-2 = 40$ pages. `sessions` outer: $\lceil 2000/40 \rceil = 50$ chunks, $2{,}000 + 50 \times 80{,}000 = 4{,}002{,}000$. `clicks` outer: $\lceil 80{,}000/40 \rceil = 2{,}000$ chunks, $80{,}000 + 2{,}000 \times 2{,}000 = 4{,}080{,}000$. Smaller outer wins by 78,000 reads. Both are terrible: the inner is rescanned 50 times.
(c) $B_R + |R| \cdot (h + \text{matches}) = 2{,}000 + 100{,}000 \times (3 + 40) = 4{,}302{,}000$. *Worse* than the block nested loop. The surprise: every session has matches, so the index sends us to every click in the table, one random page read each, 4,000,000 of them; the block loop reads the same 80,000 pages sequentially, just 50 times over. An index on the inner only pays when the outer is small or selective.
(d) Build on `sessions`, 2,000 pages, against 42 buffers: it does not fit, so Grace: $3 \times (2{,}000 + 80{,}000) = 246{,}000$. To fit, `work_mem` must hold 2,000 pages = 16 MB (plus hash overhead; 32 MB is safe). Then the cost is $2{,}000 + 80{,}000 = 82{,}000$, each input read once.
(e) Sort `sessions`: runs $\lceil 2000/42 \rceil = 48$, fan-in 41, $41 < 48 \leq 1681$ so 2 merge passes, 3 passes total, $2 \times 2{,}000 \times 3 = 12{,}000$. Sort `clicks`: runs $\lceil 80{,}000/42 \rceil = 1{,}905$, $41^2 = 1681 < 1905 \leq 41^3$ so 3 merge passes, 4 passes total, $2 \times 80{,}000 \times 4 = 640{,}000$. Merge: $2{,}000 + 80{,}000 = 82{,}000$. Total $12{,}000 + 640{,}000 + 82{,}000 = 734{,}000$.
(f) Ranking with $M = 42$: hash (Grace) 246,000 < sort-merge 734,000 < block nested loop 4,002,000 < index nested loop 4,302,000 < simple nested loop 8×10^9. With enough memory the hash join drops to 82,000. Now `WHERE s.id < 100`: 100 rows on 2 pages. Index nested loop: $2 +$

$100 \times 43 = 4{,}302$. Block nested loop: $\lceil 2/40 \rceil = 1$ chunk, $2 + 80{,}000 = 80{,}002$. Hash join: build 2 pages fits, $2 + 80{,}000 = 80{,}002$. The index nested loop wins by a factor of about 19, because now the index lets us *avoid* reading most of `clicks`; the other two must read all 80,000 pages no matter how small the outer is. Same algorithms, different sizes, different winner: that is why all three stay.

Solution: C3. Reading a plan

(a) Seq Scan on `sessions` (streams) → Hash (*breaker*: builds the table) → Seq Scan on `clicks` (streams) → Hash Join (probe streams) → HashAggregate (*breaker*) → Sort (*breaker*, but 18 rows). The costly breaker is the Hash build.
(b) The 100,000-row build side exceeded `work_mem` (4,096 KB used), so PostgreSQL ran a Grace hash join with 4 partitions: both inputs were written to disk in 4 pieces and joined pair by pair, roughly three reads and writes of each input instead of one. Raise `work_mem` to about 16 MB or more (the build is 100,000 rows; at 4 MB it needed 4 batches, so about 4×4 MB) and `Batches` drops to 1; the join cost falls from about $3(B_R + B_S)$ to $B_R + B_S$, roughly a third.
(c) A Hash node emits nothing until the whole build is done, so first and last output coincide: 61.2 ms is when the table was complete. The join can only start probing after that, so its first output row appears just after, at 61.4 ms. The gap 61.4..452.7 is the probe streaming 4 million clicks.
(d) 18 groups (one per country). The hash table has 18 entries, 40 KB, so a HashAggregate is trivially cheap and streams the 4 million join rows through it. A sort of 4 million rows just to find 18 groups would be an external merge costing far more. The planner picks sort-based grouping only when the group count is large or the input is already ordered.
(e) Expect a Merge Join. `sessions` can be delivered in `id` order for free by an Index Scan on its primary key; `clicks` has no index on `session_id` here, so a Sort node appears above its Seq Scan, and with 4 million rows it will read `Sort Method: external merge Disk: ...kB`. Slower: the join cannot start until that external sort finishes, and the sort itself costs several passes over the clicks pages.
(f) A nested loop with an index on the inner: `Nested Loop` over `Seq Scan on sessions` with `Index Scan on clicks ... (loops=100000)`. The inner index scan ran once per session. Sensible only for a small outer; for all 100,000 sessions it is the slowest option (C2(c)).

Solution: C4. Duplicates and groups

(a) Sort by `user_id` and drop adjacent equals, or hash on `user_id` and keep one entry per value. 50 distinct users: hash, a 50-entry table, one streaming pass over 4 million rows (`HashAggregate` or `Unique` over hash). 3 million distinct: a hash table of 3 million entries would blow `work_mem`, so sort then `Unique`, which keeps only the current value in memory, at the

price of an external sort.
(b) The index delivers rows in `session_id` order, so the planner uses an Index Scan feeding a `GroupAggregate`: no sort needed. It is not a pipeline breaker: each group's count is emitted when the key changes. Memory is one group's running count, regardless of how many sessions exist.
(c) Sorted input guarantees that all rows of a group are adjacent, so once the key changes the group is finished and can be emitted and forgotten; the operator holds one group. Hashed input arrives in no order, so a group is not known to be complete until the last input row, and every group's running aggregate must stay in the table until the end. With groups nearly as numerous as rows the table is nearly the input itself, so the planner sorts instead.

Solution: C5. Discussion

(a) $B = 30$, $M = 5$: runs $\lceil 30/5 \rceil = 6$, fan-in 4, $4 < 6 \leq 16$ so 2 merge passes, 3 passes, I/O $2 \times 30 \times 3 = 180$. $M = 6$: runs 5, fan-in 5, one merge pass, 2 passes, I/O 120. One third less. Each pass costs $2B$ regardless of M, so what the extra buffer buys is not a slightly faster merge but the disappearance of an entire pass, and a pass is a complete read and write of the file.
(b) (1) Memory: with a large build side the join becomes Grace and costs $3(B_R + B_S)$, as in C2(d) where it cost 246,000 rather than 82,000. (2) Indexes and sizes: with a tiny or selective outer and an index on the inner, index nested loop reads only the matching pages, as in C2(f) where it cost 4,302 against the hash join's 80,002. (3) Join condition: an inequality or range join cannot use hashing at all, and sorted inputs let sort-merge skip the expensive part so that it beats a hash join that must still build a table.
(c) Nested loop: best with a tiny outer or a selective index on the inner (C2(f)); disaster with two large inputs and nothing tiny or indexed (C2(b), 4 million reads). Sort-merge: best when both inputs arrive sorted (index scans or a previous sort) or sorted output is wanted; disaster when neither is sorted, since two external sorts come first (C2(e), 652,000 of its 734,000 reads are sorting). Hash: best for a large equi-join with no useful order and enough memory (C2(d), 82,000); disaster on an inequality join, which it cannot run, or with a build side far larger than memory, which triples its cost. Each is best exactly where another fails, and the optimiser can estimate which case it is in, so all three stay.

Summary: If you missed these, reread

- A1–A4, B4, C4: Sections 2–3 (iterator model, breakers) and 6 (aggregation). A5–A8, B1, B2, C1, C5(a): Section 4 (external sort), P1, P2.
- A9–A15, B3, B5–B7, C2, C5(b), C5(c): Section 5 (the three joins, "which join wins", "when each one is a disaster").
- A16–A20, B8, C3: Section 7 (reading a plan, forcing alternatives) and the Lab 4 companion.